

1. Introduccion

Context-Augmented Generation (CAG) es la evolucion conceptual de RAG que enfatiza contexto persistente y estructurado del USUARIO (no conocimiento general). Mientras RAG recupera de una base documental general, CAG incorpora preferencias, historial y decisiones del usuario para personalizar respuestas.

CAG no tiene un paper fundacional unico. Su fundamento teorico se encuentra en 'Building LLM Powered Applications' (Valentina Alto, O'Reilly) y 'Design Patterns for Large Language Models' (O'Reilly). Estas obras documentan patrones de memoria: episodica (conversaciones pasadas), semantica (perfil del usuario) y de trabajo (estado actual de la sesion).

1.1 RAG vs CAG

- RAG: conocimiento publico/general desde base documental estatica.
- CAG: contexto privado/especifico del usuario, dinamico durante la interaccion.
- RAG responde factualmente; CAG personaliza segun el usuario.

2. Estado Actual en el Proyecto

context_store.py (placeholder - PENDIENTE de implementar):

```
class ContextStore:
    def save(self, user_id, key, value):
        raise NotImplementedError("CAG context storage is not implemented yet")

    def list_for_user(self, user_id):
        raise NotImplementedError("CAG context retrieval is not implemented yet")
```

cag.py (placeholder - PENDIENTE de implementar):

```
def apply_context(user_id, question, base_answer, context_items):
    return base_answer
```

2.1 Lo Que el Examen Pide Implementar

Segun las pruebas de validacion (test_cag_contract.py), CAG debe:

- POST /api/context -> guardar clave-valor por usuario (status 201, saved=true).
- GET /api/context?user_id=X -> recuperar contexto (status 200, lista de items).
- POST /api/ask con contexto previo -> respuesta influenciada por contexto (context_used con la clave).

2.2 Estrategia de Implementacion Sugerida

Paso 1: context_store.py - almacen en memoria

```
class ContextStore:
    def __init__(self):
        self._data = {} # {user_id: {key: value}}

    def save(self, user_id, key, value):
        if user_id not in self._data:
            self._data[user_id] = {}
        self._data[user_id][key] = value
        return True

    def list_for_user(self, user_id):
        items = self._data.get(user_id, {})
        return [{"key": k, "value": v} for k, v in items.items()]
```

Paso 2: cag.py - inyectar contexto en respuesta

```
def apply_context(user_id, question, base_answer, context_items):
    if not context_items:
        return base_answer
    ctx_str = "; ".join(
        f"{c['key']}: {c['value']}" for c in context_items
    )
    return f"{base_answer} [Contexto: {ctx_str}]"
```

3. Patrones de Memoria para CAG

- Memoria Episodica: historial de preguntas y respuestas anteriores del usuario.
- Memoria Semantica: preferencias del usuario (nivel, formato, temas de interes).
- Memoria de Trabajo: estado transitorio de la sesion actual.

Para una implementacion completa, context_store podria persistir a disco (JSON/ SQLite) en lugar de solo memoria volatil.

4. Referencias

Building LLM Powered Applications (Valentina Alto) - O'Reilly

Design Patterns for Large Language Models - O'Reilly

ACM Digital Library - 'context aware systems', 'software traceability'

arXiv - LLM Context & Memory

O'Reilly - LLM Architecture and Design Patterns